

Progettare Hardware

Progettare Hardware

Questo capitolo si focalizza sulle porte fondamentali, chiamate Boolean Gates.

Dato che il loro utilizzo implementa funzioni booleane, è necessario essere a conoscenza dell'algebra di Boole per poterle utilizzare nel modo corretto.

L'algebra booleana gestisce i valori binari (vero/falso, 1/0, yes/no ...).
Una Funzione Booleana è una funzione che opera con valori di input binari e restituisce output binari.

Le funzioni booleane giocano un ruolo chiave nella costruzione, ottimizzazione hardware.

Avere confidenza con tali funzioni è il primo passo fondamentale per capire nel dettaglio il funzionamento di una CPU.

Algebra Booleana

Esistono tre rappresentazioni di una funzione booleana.

- Tabella di verità
- Espressione Booleana
- Forma canonica

Algebra Booleana

Tabella di verità: è il modo più facile di rappresentare una funzione booleana e consiste nel tracciare una tabella di tutte le possibili combinazioni tra gli input e associare un output ad ogni combinazione.

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Algebra Booleana

Espressione Booleana: consiste in un'espressione matematica che fa uso dei di operatori booleani.

Gli operatori booleani sono "And", "Or" e "Not".

- All'operatore And associamo il *
- All'operatore Or associamo il +
- All'operatore Not associamo il trattino sopra alla variabile, esempio: $\bar{x}$

Esempio:

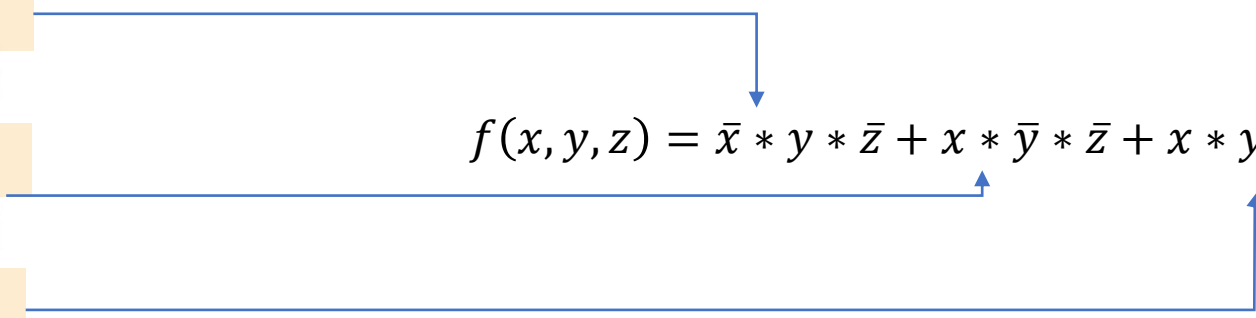
$$f(x, y, z) = \bar{x} * y * \bar{z} + x * \bar{y} * \bar{z} + x * y * \bar{z}$$

Algebra Booleana

E' possibile passare da tabella di verità a Espressione Booleana facendo i seguenti passaggi:

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

1. Individuo le righe che hanno 1 come output
2. Moltiplico tra di loro le variabili di input di quella riga negando le variabili a zero
3. Sommo la riga successiva

$$f(x, y, z) = \bar{x} * y * \bar{z} + x * \bar{y} * \bar{z} + x * y * \bar{z}$$


Algebra Booleana

E' possibile passare da tabella di verità a Espressione Booleana facendo i seguenti passaggi:

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

1. Individuo le righe che hanno 1 come output
2. Moltiplico tra di loro le variabili di input di quella riga negando le variabili a zero
3. Sommo la riga successiva

$$f(x, y, z) = \bar{x} * y * \bar{z} + x * \bar{y} * \bar{z} + x * y * \bar{z}$$

4. Posso rimuovere il * esattamente come nelle espressioni matematiche

$$f(x, y, z) = \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z}$$

Algebra Booleana

E' possibile passare da Espressione Booleana a tabella di verità partendo dall'espressione, inserendo una riga di valori di input e trovando il risultato:

$$\begin{aligned} f(x, y, z) &= \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z} = \\ &= \bar{0}0\bar{0} + 0\bar{0}\bar{0} + 00\bar{0} = \\ &= 101 + 011 + 001 = \\ &= 0 + 0 + 0 = \\ &= 0 \end{aligned}$$

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Algebra Booleana

E' possibile passare da Espressione Booleana a tabella di verità partendo dall'espressione, inserendo una riga di valori di input e trovando il risultato:

$$\begin{aligned} f(x, y, z) &= \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z} = \\ &= \bar{0}0\bar{1} + 0\bar{0}1 + 00\bar{1} = \\ &= 100 + 011 + 000 = \\ &= 0 + 0 + 0 = \\ &= 0 \end{aligned}$$

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Algebra Booleana

E' possibile passare da Espressione Booleana a tabella di verità partendo dall'espressione, inserendo una riga di valori di input e trovando il risultato:

$$\begin{aligned} f(x, y, z) &= \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z} = \\ &= \bar{0}1\bar{0} + 01\bar{0} + 01\bar{0} = \\ &= 111 + 011 + 001 = \\ &= 1 + 0 + 0 = \\ &= 1 \end{aligned}$$

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Algebra Booleana

Un'espressione Booleana viene definita in forma canonica quando viene espressa sotto forma di:

- Somme di prodotti $f(x, y, z) = \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z}$
- Prodotto di somme $f(x, y) = (\bar{x}y)(x\bar{y})$

$$f(x, y, z) = \bar{x}y\bar{z} + x(\bar{y}\bar{z} + y\bar{z}) \quad \text{Espressione Booleana}$$

$$f(x, y, z) = \bar{x}y\bar{z} + x\bar{y}\bar{z} + xy\bar{z} \quad \text{Forma Canonica}$$

Algebra Booleana

Proprietà dell'Algebra Booleana:

- Identità: $x+0=x$, $x*1=x$
- Assorbimento: $x+1=1$, $x*0=0$
- Inverso: $x + \bar{x} = 1$, $x * \bar{x} = 0$
- Idempotenza: $x + x = x$, $x * x = x$
- Commutativa: $x + y = y + x$, $xy = yx$
- Associativa: $x + (y + z) = (x + y) + z$, $x * (y * z) = (x * y) * z$
- Distributiva: $x * (y + z) = (x * y) + (x * z)$
- Doppia negazione: $\bar{\bar{x}} = x$
- Assorbimento:
 - $x + \bar{x}y = x + y$
 - $x * (\bar{x} + y) = x * y$
 - $x * (x + y) = x$
 - $x + xy = x$

Algebra Booleana

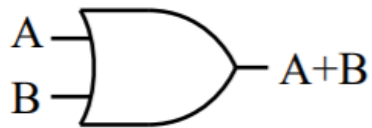
Teoremi di De Morgan:

- $\overline{x + y} = \bar{x} * \bar{y}$
- $\overline{x * y} = \bar{x} + \bar{y}$

Tali teoremi sono generalmente utilizzati per la semplificazione di una funzione booleana.

Porte Logiche

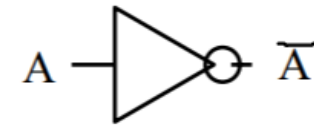
Porte Logiche primitive



OR		
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1



AND		
A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

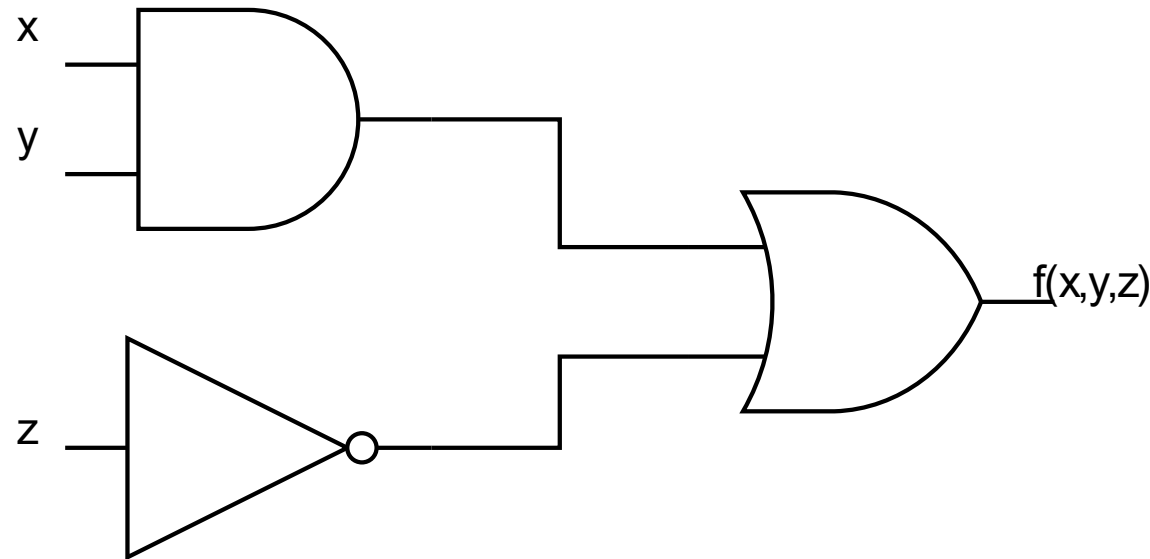


NOT	
A	$\overline{A}$
0	1
1	0

Algebra Booleana e Porte Logiche

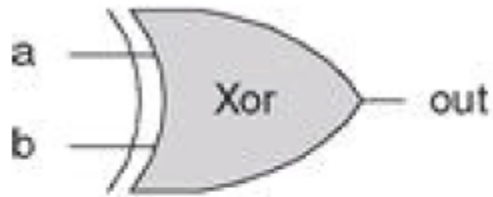
E' possibile convertire un'espressione booleana o una forma canonica nel circuito equivalente formato da porte logiche.

Esempio: $f(x, y, z) = xy + \bar{z}$



Porte Logiche

Porta Xor: è una delle porte più importanti, ha la seguente tabella di verità ed è alla base dei circuiti sommatori.



a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Porte Logiche e Algebra Booleana

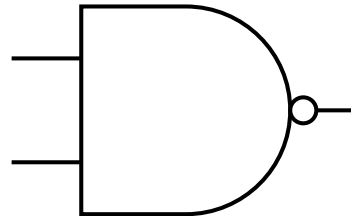
Esercizi proposti:

Categoria 1: data la seguente tabella di verità, scrivere la funzione sottoforma di equazione booleana e poi creare il circuito logico utilizzando porte AND, OR e NOT.

Categoria 2: dato il seguente circuito, scrivere la relativa equazione booleana, applicare una semplificazione secondo le regole dell'algebra booleana, infine disegnare il nuovo circuito semplificato.

Porte Logiche

Porta Nand: tale porta è particolarmente interessante perché, data la sua struttura interna, consuma energia solamente al cambio del valore di output. Questa porta è alla base di tutta la costruzione del processore ed ha la seguente tabella di verità (praticamente è una AND con l'uscita negata):

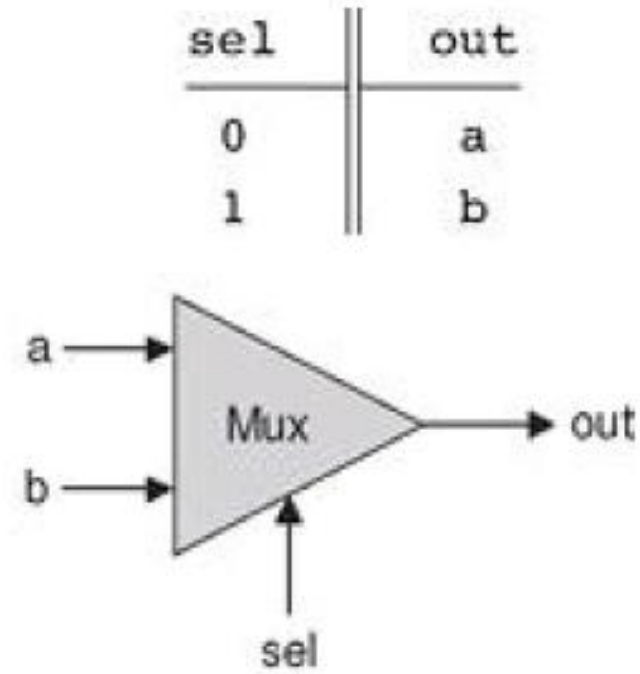


a	b	$\text{Nand}(a, b)$
0	0	1
0	1	1
1	0	1
1	1	0

Porte Logiche

Multiplexer: porta a tre input che utilizza un pin per decidere quale degli ingressi sarà utilizzato come output.

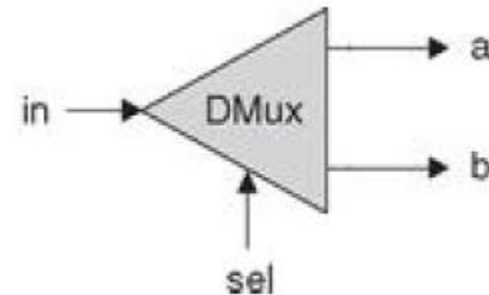
a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1



Porte Logiche

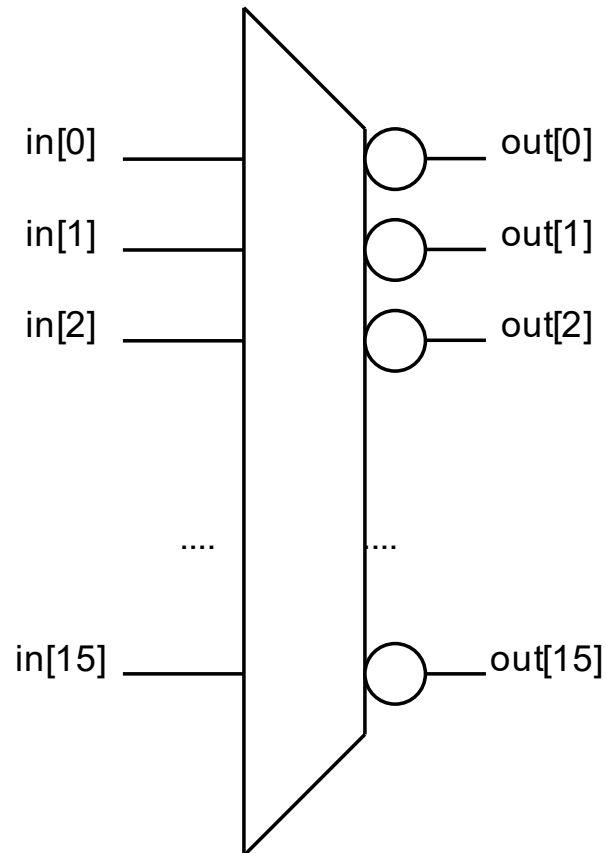
Demultiplexer: dispositivo con due pin di ingresso che serve per selezionare su quale output (in questo caso chiamati 'a' e 'b') manderà il segnale di input.

sel	a	b
0	in	0
1	0	in

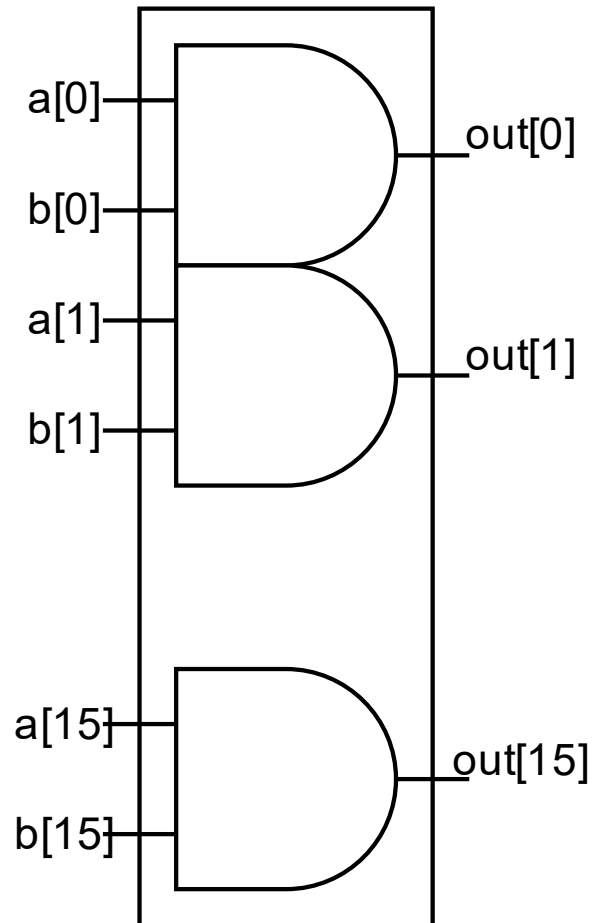


Porte Logiche

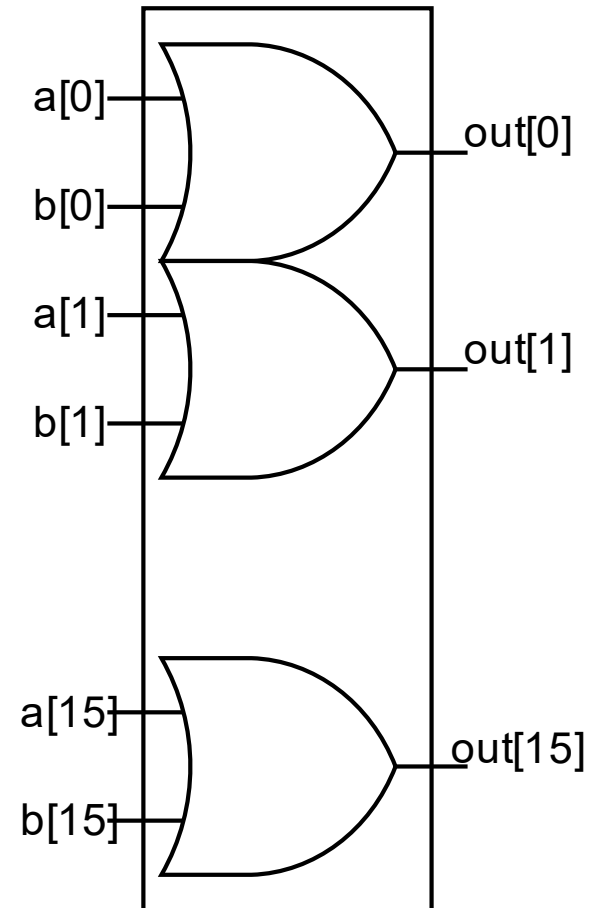
Multi bit Not:



Multi bit And:



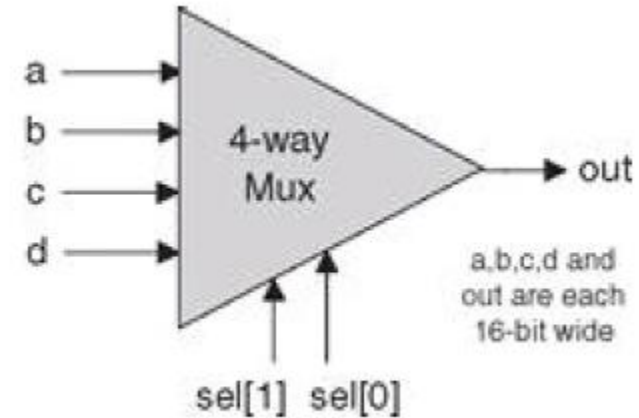
Multi bit Or:



Porte Logiche

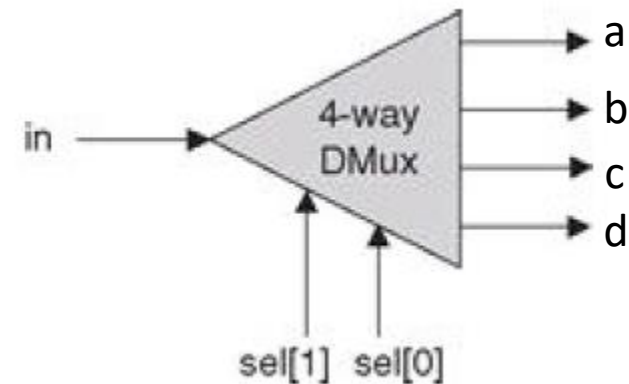
Multi bit Multiplexer:

sel[1]	sel[0]	out
0	0	a
0	1	b
1	0	c
1	1	d



Multi-Way Gates

sel[1]	sel[0]	a	b	c	d
0	0	in	0	0	0
0	1	0	in	0	0
1	0	0	0	in	0
1	1	0	0	0	in



Progetto

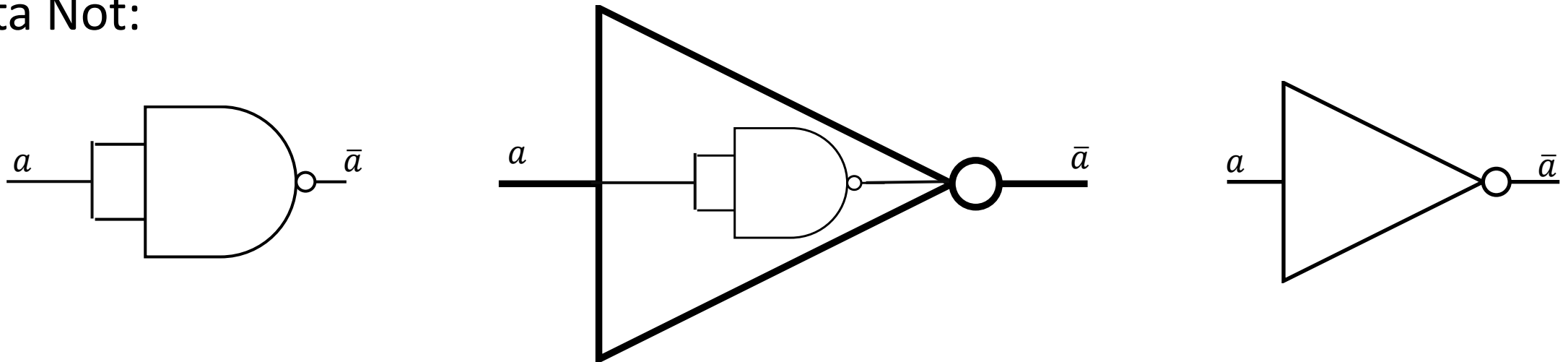
Obiettivo: implementare tutte le porte logiche viste a lezione utilizzando solamente la porta logica NAND e tutte le successive porte che verranno gradualmente implementate.

Strumenti: le porte logiche dovranno essere implementate in linguaggio HDL, simulate e testate con l'hardware simulator.

Progetto

E' possibile costruire le porte AND OR e NOT partendo dalla porta NAND.

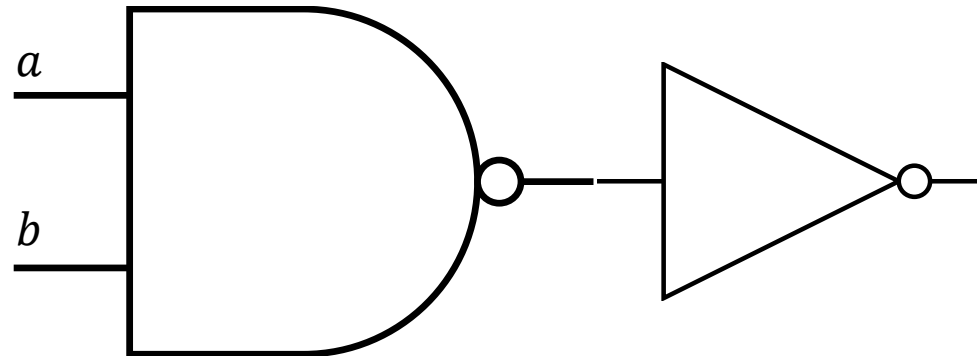
Porta Not:



Progetto

A questo punto possiamo creare la porta AND facendo uso della porta NAND e della porta NOT.

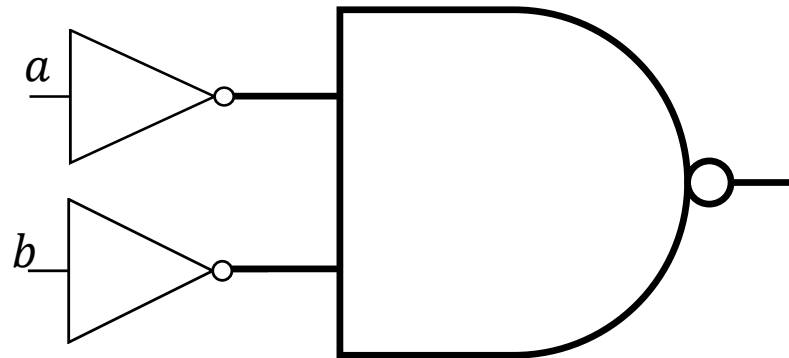
Porta Not:



Progetto

Allo stesso modo creiamo la porta OR e facciamo uso del teorema di De Morgan.

Porta Or:

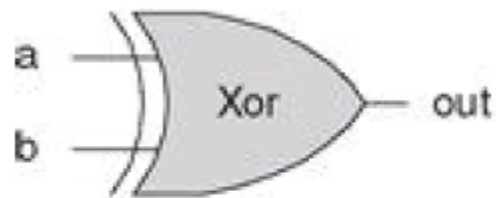


Per il teorema di De Morgan si ottiene che

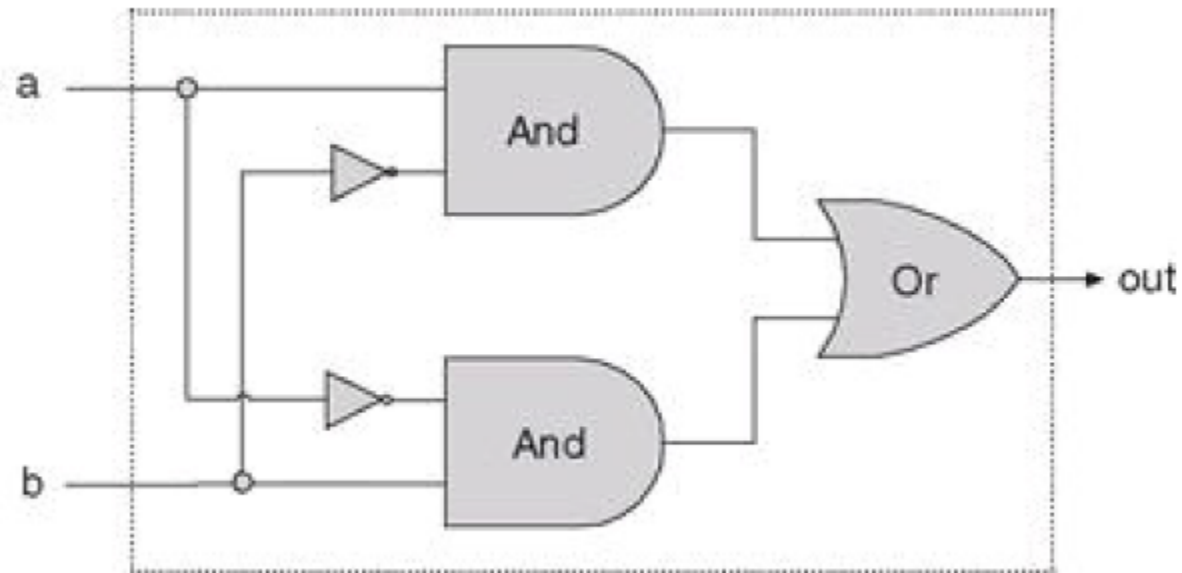
$$\overline{\overline{a} * \overline{b}} = \overline{\overline{a}} + \overline{\overline{b}} = a + b$$

Progetto

Porta Xor: è una delle porte più importanti, ha la seguente tabella di verità ed è alla base dei circuiti sommatori.



a	b	out
0	0	0
0	1	1
1	0	1
1	1	0



Progetto

Implementare le seguenti porte logiche in linguaggio HDL:

- Not
- And
- Or
- Xor
- Multiplexer
- Demultiplexer
- Multi bit Not
- Multi bit And
- Multi bit Or
- Multi bit Multiplexer
- Multi-way Gates